grpcshop: gRPC Microservices

Project report: what it is, what was built, and how well it works

- **Project:** grpcshop, an order-processing system built from gRPC microservices in Go and Python
- **Repository:** github.com/Dhagash1206/gRPC-Microservice
- **Date:** 1 November 2026
- **Status:** Built, running and verified; all 153 automated tests pass, and CI runs them on every push

M1. Project Summary

> **Aim:** Show how microservices talk to each other over gRPC under production conditions: mutual TLS between services, a JWT-protected public edge, crash-safe distributed transactions, asynchronous events and full observability.

What the project is about

grpcshop is an online-shop checkout. Customers browse products and place orders through a single HTTPS endpoint. Behind it, four services split the work: Order runs the checkout, Inventory holds stock, Payment charges the customer, and Notification emails a confirmation. Each service owns its own database, and they never share tables.

Because no single database spans the services, a checkout is a saga: a chain of local steps, each with an undo step. Every step is recorded, so a failure or crash at any point is either undone or finished automatically.

At a glance

Item	Value
Services	4 application services (2 Go, 2 Python) plus an Envoy edge
Containers	15, started with one command (make up)
Public API	6 REST endpoints, also callable as native gRPC
Internal RPCs	13 across 4 versioned proto contracts
Databases	4 PostgreSQL instances, one per service, TLS-only
Tests	68 Go, 48 Python integration, 37 end-to-end (all 153 pass)
CI	GitHub Actions: lint, vulnerability scan, all three test suites

Technology choices

Area	Choice	Why
Service calls	gRPC with Protocol Buffers	Typed contracts, HTTP/2, deadlines and retries built in
Languages	Go (Order, Notification), Python (Inventory, Payment)	Shows two languages calling each other from shared protos
Edge	Envoy proxy	TLS termination, JWT checks, REST-to-gRPC transcoding, rate limiting
Data	PostgreSQL, one database per service	Each service owns its data
Async events	RabbitMQ quorum queues	Durable delivery with dead-lettering
Email	SMTP to Mailpit (dev inbox)	Real emails that tests can inspect
Observability	OpenTelemetry, Jaeger, Prometheus, Grafana	Traces, metrics, alerts and dashboards
Deployment	Docker Compose	One command runs the whole system
CI	GitHub Actions	Every push is linted, scanned and tested

M2. Architecture

Services

Service	Language	Responsibility	Allowed callers
Envoy (edge)	Config	Public HTTPS on port 8443; JWT checks; REST and gRPC entry point	Any client
Order	Go	Runs the checkout saga; stores orders and the event outbox; recovery worker	envoy (plus dev-client in dev)
Inventory	Python	Products, stock holds with expiry, admin restock	order, envoy
Payment	Python	Charges, refunds and cancellation fences	order
Notification	Go	Consumes order events and sends emails	None (event consumer)

How a new order flows

1. The client sends POST /v1/orders with a JWT to Envoy over HTTPS.
2. Envoy checks the JWT, converts the JSON request to gRPC and calls Order over mutual TLS.
3. Order reads prices from Inventory, saves the order as PENDING, reserves stock, charges Payment and commits the reservation.
4. Order marks the order CONFIRMED and writes an OrderConfirmed event to its outbox in the same database transaction.
5. A background relay publishes the event to RabbitMQ; Notification consumes it and emails the customer.

Network segmentation

Each database sits on its own internal Docker network that only its owning service joins, so a compromised service cannot reach another service's database.

Network	Members	Purpose
public	Envoy, dev UIs	Only network with published host ports
edge	Envoy, Order, Inventory	Edge to services
services	Order, Inventory, Payment	Service-to-service gRPC
events	Order, Notification, RabbitMQ	Async messaging
***-data (x4)**	Each service and its own database	Database isolation
mail	Notification, Mailpit	SMTP
observability	All services, Collector, Jaeger, Prometheus, Grafana	Telemetry export and scraping
broker-metrics	Prometheus, RabbitMQ	RabbitMQ metrics scraping only

> **Fixed:** RabbitMQ used to join the shared observability network for metrics scraping, which let Inventory reach the broker. It now uses a dedicated broker-metrics network shared with Prometheus only, and the isolation test passes.

M3. Service-to-Service Communication

Summary: All internal calls are gRPC over mutual TLS 1.3. Every service proves its identity with a certificate, and each service accepts calls only from a named list of callers.

Patterns used

Concern	Implementation
Contracts	Versioned proto packages (shop.*.v1); old fields are reserved, never reused
Connections	Long-lived channels with keepalive; automatic retries on UNAVAILABLE
Timeouts	Each downstream call has a 2 s timeout that never outlasts the caller's deadline
Safe retries	Every write is keyed (order ID, idempotency key), so a repeat has no extra effect
Error mapping	Business errors pass through; infrastructure failures become UNAVAILABLE without internal details
Input validation	SKUs must be 1-64 characters of [A-Za-z0-9._-]; errors never echo unbounded input
Caller authorization	Per-service allowlist of certificate names (CN); health checks are exempt
Request tracing	Envoy's x-request-id is logged by every service and returned to the client
Load protection	1 MiB message cap; bounded concurrent RPCs fail fast with RESOURCE_EXHAUSTED
Public API	Envoy turns REST/JSON into gRPC using google.api.http annotations; gRPC clients use the same endpoint

Public API

Method and path	RPC	Access
GET /v1/products	InventoryService.ListProducts	Public
POST /v1/orders	OrderService.CreateOrder	JWT
GET /v1/orders	OrderService.ListOrders (paginated)	JWT
GET /v1/orders/{id}	OrderService.GetOrder	JWT, own orders only
POST /v1/orders/{id}:cancel	OrderService.CancelOrder	JWT, own orders only
POST /v1/products/{sku}:restock	InventoryService.RestockProduct	JWT with admin role

Internal RPCs

Service	RPCs
Inventory	BatchGetProducts, ListProducts (server stream), ReserveStock, CommitReservation, ReleaseStock, RestockProduct
Payment	Charge, CancelCharge
Order	CreateOrder, GetOrder, ListOrders, CancelOrder
Events	OrderConfirmed, OrderCancelled (RabbitMQ messages, not RPCs)

M4. Checkout Saga and Reliability

> **Key note:** Order saves its progress before every remote call, so whatever happens, the order can be finished or undone later.

Checkout steps

1. Get prices from Inventory and save the order as PENDING.
2. Reserve stock (the hold expires after 15 minutes unless committed).
3. Charge the payment, keyed by the order ID.
4. Commit the stock reservation.
5. Mark the order CONFIRMED and write the OrderConfirmed event to the outbox in one transaction.

What happens when a step fails

Failure point	Result	Undo performed
Out of stock	Order FAILED, error returned	None (nothing was reserved)
Reserve timed out	Order FAILED	Release stock (blocks a late reservation)
Payment declined	Order FAILED	Release stock
Charge timed out	Order FAILED	Cancel charge (refund or block) and release stock
Stock hold expired	Order FAILED	Refund payment and release stock
Order service crashed	Recovery worker finishes it	Cancel charge and release stock, then FAILED

Reliability mechanisms

- **Fences against late calls**
  - **Payment:** Cancelling a charge that does not exist yet stores a cancelled record, so a delayed charge with that key is rejected.
  - **Inventory:** Releasing an order with no reservation stores a released record, so a delayed reservation is rejected.
- **Recovery worker**
  - **Trigger:** Orders stuck in PENDING, COMPENSATING or CANCELLING for over 60 seconds.
  - **Safety:** Claims orders with a database lease, so several replicas never handle the same order.

- **Transactional outbox**
  - **Write:** The event is saved in the same transaction as the order, so it is never lost.
  - **Publish:** A relay sends events with broker confirmations and backs off up to 10 seconds while RabbitMQ is down.
  - **Unroutable events:** Events the broker cannot route stay pending and are retried, instead of counting as sent.
- **Client idempotency key**
  - **Problem solved:** A client retry after a timeout no longer creates a second order and charge.
  - **Behaviour:** A repeat returns the original order; a different request with the same key is rejected.
- **Notification delivery**
  - **Duplicates:** Processed event IDs are stored in Notification's own database.
  - **Outages:** Retries with backoff (about 61 s per delivery) before requeueing; failures go to a dead-letter queue.
  - **Replay:** make dlq-replay moves dead-lettered messages back, acknowledged only after the broker confirms.

M5. Security

Protection by layer

Layer	Protection
Client to edge	HTTPS; RS256 JWT checked for signature, issuer, audience and expiry
Edge	Per-client-IP rate limit; 64 KiB body limit; CORS allowlist; security headers; unknown query parameters rejected
Service to service	Mutual TLS 1.3 with per-service caller allowlists; Envoy pins each upstream's certificate name
Databases	TLS 1.3 only; services log in with certificates, so no database passwords exist
Message broker	TLS 1.3 only; certificate login (EXTERNAL); least-privilege permissions per service
Email	STARTTLS required
Admin actions	Restock needs the admin role, checked by Envoy and again by Inventory
Telemetry	Services export to the OpenTelemetry Collector over mTLS
Dependencies	govulncheck runs in CI; golang.org/x/net upgraded to v0.60.0

Loopholes found and fixed

Each problem was reproduced against the running system before it was fixed, then re-tested.

Problem	Fix
Client retry after a gateway timeout could charge twice	Client idempotency key; repeats return the original order
Quantities 2147483647 + 2147483647 + 3 wrapped to 1 (int32 overflow)	Per-item limit of 1,000 and at most 50 products per order
5 MB request bodies reached the services	Envoy rejects bodies over 64 KiB with HTTP 413
5,000 line items reached Inventory; the error echoed every SKU	Item-count limits; error lists 10 SKUs then "+N more"
No rate limiting (300 of 300 burst requests accepted)	Per-client-IP token buckets plus an instance-wide cap
Envoy admin API reachable from other containers	Admin bound to loopback; only /stats/prometheus exposed
A 4-second SMTP outage dead-lettered emails	In-process retry with backoff; a 20-second outage now loses nothing
Replayed dead letters would be dead-lettered again	Dispatch on message type instead of routing key
Order crash left stock held with no owner	Saved saga state, recovery worker and 15-minute hold expiry
JWT subject could carry email header injection (CR/LF)	Customer IDs restricted to a safe character set
A NUL byte in a SKU returned 503 and could trip the error-rate alert	SKU format validated in Order and Inventory; now 400
A 10 KB SKU was echoed back in a 10 KB error	Length limit; errors no longer repeat the SKU
Inventory could reach RabbitMQ through the observability network	Dedicated broker-metrics network for scraping

Attacks tested and rejected

- **Forged identity header:** Envoy overwrites a client-supplied x-jwt-payload header; Order also rejects duplicate values.
- **Algorithm confusion:** An HS256 token signed with the public RSA key is rejected (401).
- **Wrong audience or expired token:** Rejected with 403 and 401.
- **Cross-customer access:** Reading or cancelling another customer's order returns 404.
- **Internal RPCs via the edge:** ReserveStock and Charge are not routable (gRPC status UNIMPLEMENTED).

M6. Observability

> **Status:** Built, running and covered by automated end-to-end tests. Each checkout produces a single trace across all five services, and the tests confirm that metrics, alert rules and scrape targets are live.

Components

Component	Role	Dev address
OpenTelemetry Collector	Receives traces and metrics over mTLS; turns spans into request, error and latency metrics	Internal only
Jaeger 2.22	Trace storage and search	127.0.0.1:16686
Prometheus 3.15	Metrics and alert rules	127.0.0.1:9090
Grafana 13.2	Provisioned "grpcshop overview" dashboard	127.0.0.1:3001

Tracing

- **Coverage:** Envoy, Go (gRPC and Postgres), Python (gRPC and Postgres), the outbox relay, RabbitMQ and SMTP.
- **Across the async hop:** The trace context is saved in the outbox row and carried in message headers, so the email step joins the original request's trace.
- **Correlation:** Every response returns x-request-id; service spans carry it as request.id (Envoy as guid:x-request-id), and log lines include trace_id.

Verified trace for one order

A CreateOrder request produces one trace with spans from envoy, order, inventory, payment and notification. It contains:

- **Edge and saga:** ingress, CreateOrder, BatchGetProducts, ReserveStock, Charge, CommitReservation
- **Database:** BEGIN, SELECT, INSERT, UPDATE, COMMIT and the order_lines bulk copy
- **Async path:** publish order.confirmed, process order.confirmed, smtp send

Metrics

Group	Metric names (Prometheus)
Business	grpcshop_checkout_outcomes_total, grpcshop_saga_compensations_total, grpcshop_payment_charges_total, grpcshop_payment_charged_cents_total, grpcshop_payment_cancellations_total, grpcshop_inventory_reservations_total, grpcshop_inventory_restocked_units_total, grpcshop_notifications_total
Outbox	grpcshop_outbox_backlog, grpcshop_outbox_oldest_age_seconds, grpcshop_outbox_published_total
RPC and DB	rpc_server_call_duration_seconds, rpc_client_call_duration_seconds, db_client_operation_duration_seconds
From spans	traces_span_metrics_calls_total, traces_span_metrics_duration_milliseconds

Alert rules

Alert	Fires when
OutboxNotDraining	The oldest unpublished event is older than 60 s for 2 minutes
DeadLettersWaiting	Any dead-letter queue holds messages for 1 minute
CompensationFailing	Any saga undo step failed in the last 10 minutes
HighServerErrorRatio	A service fails more than 5% of requests for 5 minutes
TelemetryPipelineDown	The collector cannot be scraped for 2 minutes

M7. Testing and Verification

> **Result:** On 1 November 2026, all 153 automated tests passed against the running stack, with the Go race detector enabled. GitHub Actions runs the same checks on every push.

Automated test suites

Suite	Command	Result	Covers
Go unit and integration	make test-go	68 passed	Saga failure paths, recovery, cancellation, idempotency, input validation, events, telemetry; repository, outbox relay and RabbitMQ publisher against real Postgres and RabbitMQ
Python integration	make test-python	48 passed	Real Postgres: reservations, expiry, fences, concurrent overselling, payments, SKU validation, mTLS and caller rules, restock
End-to-end	make e2e	37 passed	Through Envoy: auth attacks, order lifecycle, emails, edge limits, CORS, network isolation, database TLS, one trace per order, metrics and alert rules

Coverage of previously untested packages

Package	Before	After	What the new tests prove
outbox	0%	55%	Oldest-first publishing exactly once; unconfirmed events stay pending; publish spans continue the request's trace
rabbitmq	0%	73%	Persistent messages with IDs and trace headers; unroutable events not counted as sent; reconnect after a drop; dead-letter replay

Continuous integration

Job	Steps
go	go vet, staticcheck, govulncheck, go test -race (testcontainers)
python	Generate stubs, pytest against real Postgres
e2e	Generate certificates, start all 15 containers, run the end-to-end suite, print logs on failure

Manual failure drills

These drills were run during development against the live stack.

Scenario	Result
Order killed mid-checkout (payment paused)	Recovered 6 s after restart: order FAILED, charge blocked, stock back to 10
RabbitMQ stopped during an order	Order succeeded; event delivered about 8 s after recovery
SMTP down for 20 seconds	Email delivered about 14 s after recovery; nothing dead-lettered
Burst of 1,500 requests from one client	1,061 throttled; a second client still got 10 of 10 through
23 malformed requests (bad JSON, types, IDs, tokens)	All rejected with 4xx after the SKU fix
Plaintext database and broker connections	Refused

M8. Running the Project

Requires Docker, Go 1.26+ and Python 3.12+.

Commands

Command	What it does
make setup	Python venv, Go code generators, dev certificates, generated code
make up / make down	Start (and wait for health) or stop all 15 containers
make clean	Stop and delete all data volumes
make test	Python and Go suites (Docker needed for Postgres and RabbitMQ)
make e2e	End-to-end tests through Envoy
make demo	Scripted walkthrough of the REST API
make token SUB=...	Print a dev JWT (use the script with --role admin for admin tokens)
make dlq-replay	Move dead-lettered notifications back for processing
make proto	Regenerate Python, Go and Envoy code from the protos

Local addresses

Address	Service	Exposure
https://localhost:8443	Public API (Envoy)	All interfaces
http://127.0.0.1:15672	RabbitMQ management	Localhost only
http://127.0.0.1:8025	Mailpit inbox	Localhost only
http://127.0.0.1:16686	Jaeger	Localhost only
http://127.0.0.1:9090	Prometheus	Localhost only
http://127.0.0.1:3001	Grafana	Localhost only

Try it by hand

1. Get a token: TOKEN=$(make -s token SUB=customer-42)
2. List products: GET https://localhost:8443/v1/products (pass --cacert certs/ca.crt to curl)
3. Place an order: POST /v1/orders with {"items":[{"sku":"MOUSE","quantity":2}]} and the Authorization header
4. Order a LAPTOP ($1,499.99): Payment declines charges over $1,000, which shows the release-stock path

M9. Status, Limitations and Next Steps

Implementation status

Area	Status	Evidence
Core services and saga	Done	Integration, Go and end-to-end tests pass
Cancellation, restock, pagination	Done	Covered by Go, Python and end-to-end tests
Security hardening	Done	Attack and input-validation tests pass
Network segmentation	Done	All isolation tests pass
Observability	Done	End-to-end trace and metrics tests pass
Continuous integration	Done	GitHub Actions workflow, validated with actionlint
Documentation	Done	README and this report reflect the current code

Next steps

1. Deploy to Kubernetes with a service mesh handling mTLS and certificate rotation.
2. Replace dev_auth.py with a real identity provider and the fake payment service with Stripe test mode.
3. Add a small web frontend for demos.
4. Add load tests (k6) to measure throughput, latency and rate-limit behaviour.

Known limitations

- **Risk:** Certificates, keys and dev UI passwords are for development only; production needs a secret manager and certificate rotation.
- **Risk:** Every service joins the shared observability network, so services can open connections to each other there; only the mTLS caller allowlists stop them.
- **Risk:** Telemetry inside the observability network (Collector to Jaeger, Prometheus scrapes) is not encrypted.
- **Limit:** Emails go to Mailpit; customer addresses are derived from the customer ID.
- **Limit:** Rate limits are per Envoy instance; multiple edge replicas would need a shared rate-limit service.
- **Limit:** grpcmw and tlsconfig are still covered only by the end-to-end suite.